

Gym Management System — Developer Guide

1. System Overview

The Gym Management System is a console-based Java application used to manage gym users, memberships, workout classes, and merchandise. The application uses PostgreSQL for persistent data storage and JDBC to communicate between Java and the database.

Users are assigned one of three roles: **Admin, Trainer, or Member**. After login, the user is directed to a role-specific menu containing only the features available to that role.

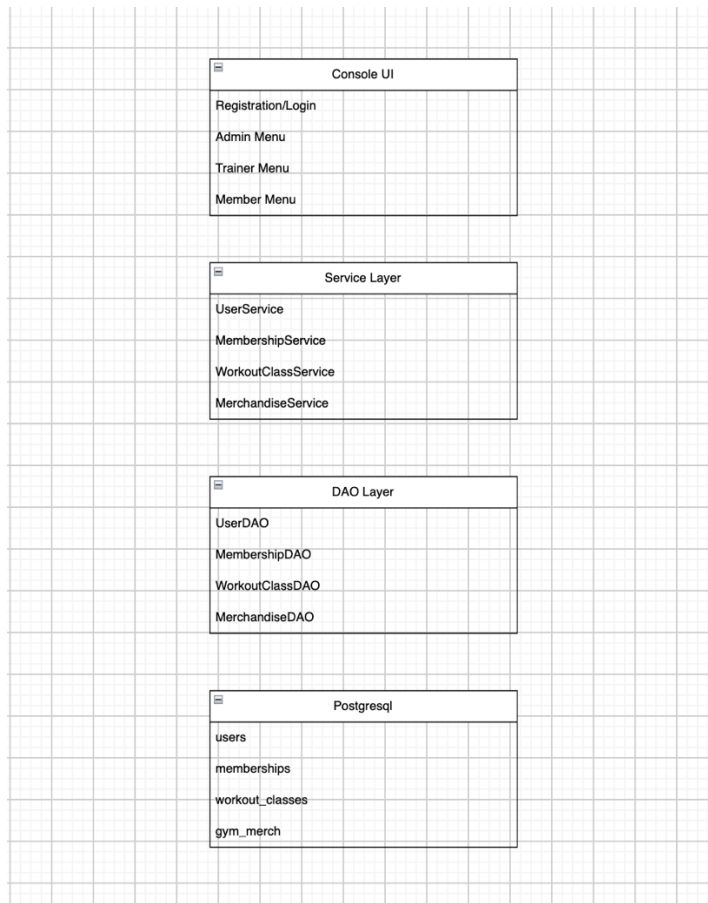
2. Architecture Overview

The application follows a layered structure:

Console UI → Service Layer → DAO Layer → PostgreSQL

The menu classes handle user input and display information. Service classes contain validation, calculations, and business logic. DAO classes handle SQL queries and database communication. Model classes represent the data moving through these layers.

Architecture Diagram



3. Class Design

The project is separated into four main feature areas:

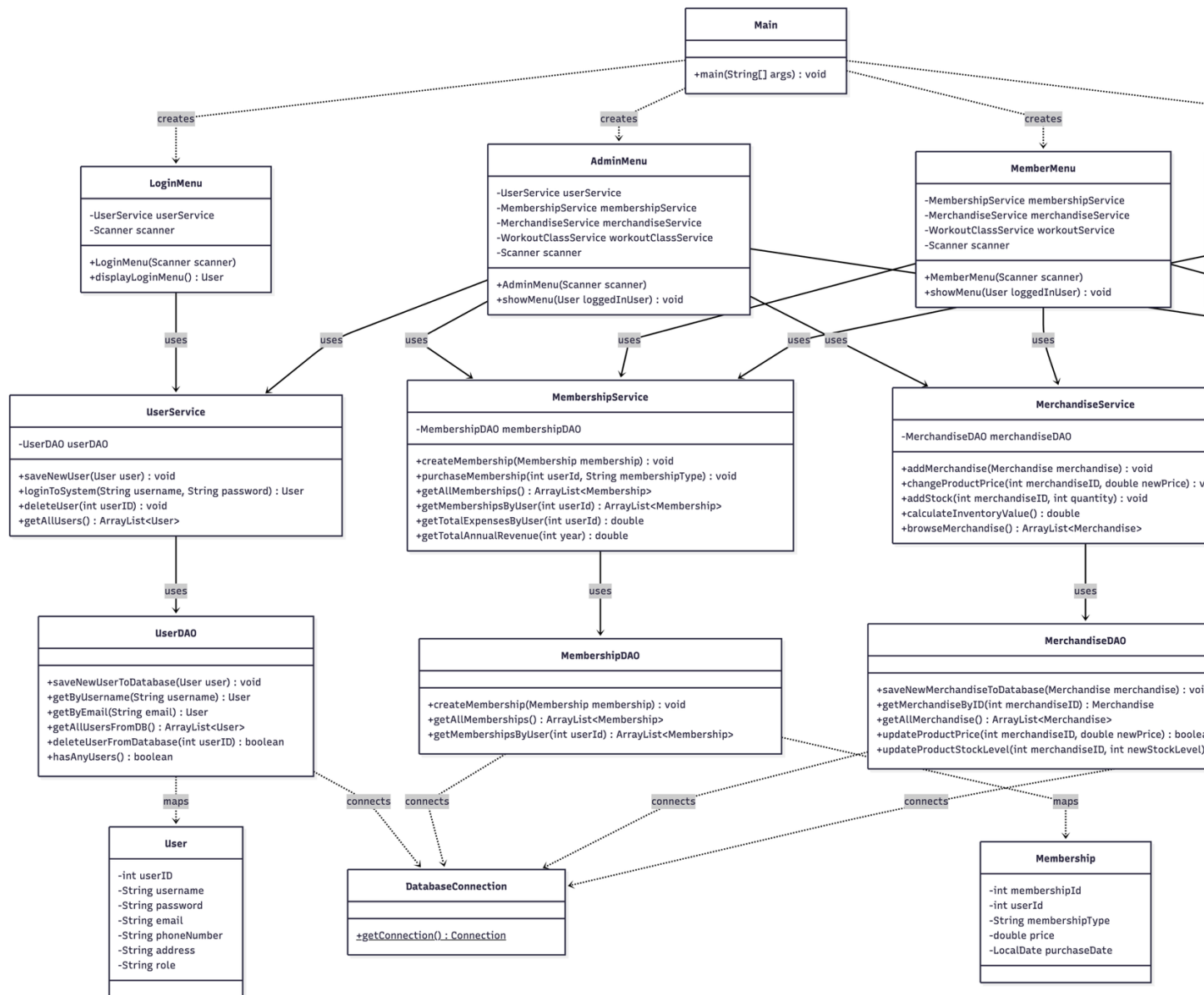
Feature	Model	Service	DAO
Users	User	UserService	UserDAO
Memberships	Membership	MembershipService	MembershipDAO
Merchandise	Merchandise	MerchandiseService	MerchandiseDAO
Workout Classes	WorkoutClass	WorkoutClassService	WorkoutClassDAO

The menu classes are `LoginMenu`, `AdminMenu`, `TrainerMenu`, and `MemberMenu`. `Main` handles application startup and directs authenticated users to the appropriate menu based on their role.

The reports package contains `MerchandiseReportExporter` and `MembershipReportExporter`, which generate the Admin text-file reports using data retrieved through the existing service layer.

The project mainly uses **composition rather than inheritance**. Services contain their corresponding DAO objects, while menus use the service classes required for their available features.

UML Class Diagram



4. Role-Based Access

The application uses separate menus to control what each role can access.

Admin users can manage users, merchandise and workout classes, view membership revenue, and export reports.

Trainer users can manage workout classes, view their assigned classes, purchase/view memberships, and browse merchandise.

Member users can purchase memberships, browse merchandise and workout classes, and view their personal membership expenses.

The logged-in `User` object is passed from `Main` to the appropriate menu so actions can be connected to the correct user.

5. User Authentication

Registration and login are handled through `LoginMenu`, `UserService`, and `UserDAO`.

Passwords are hashed using **BCrypt** before being stored in PostgreSQL. During login, BCrypt compares the entered password with the stored hash rather than storing or comparing plain-text passwords.

The authentication flow is:

`LoginMenu` → `UserService` → `UserDAO` → `users` table

6. Database Design

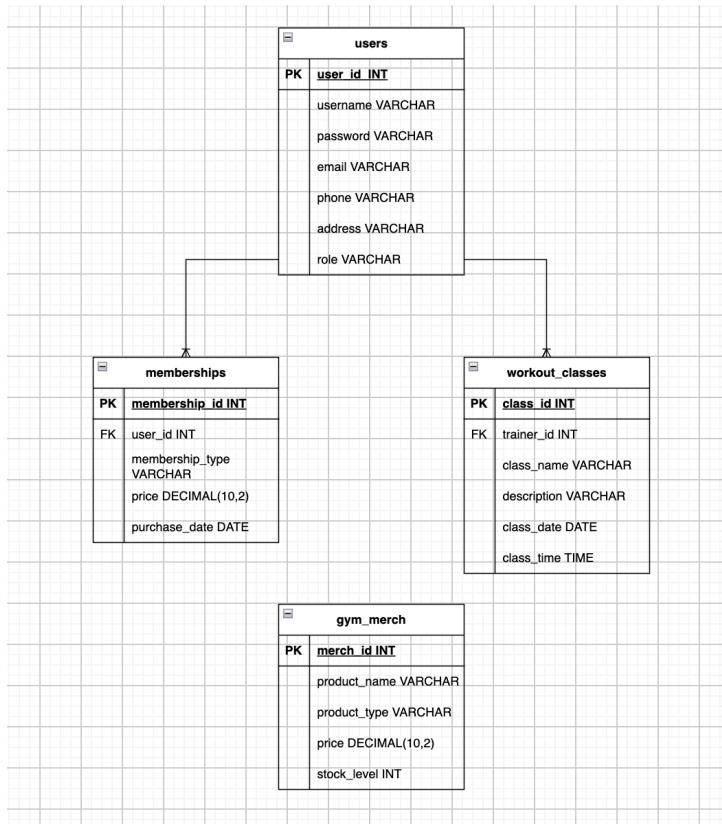
The application uses four PostgreSQL tables:

Table	Purpose
<code>users</code>	Stores user credentials, contact information and roles
<code>memberships</code>	Stores membership purchases
<code>workout_classes</code>	Stores workout classes and their assigned trainer
<code>merchandise</code>	Stores merchandise and inventory information

`memberships.user_id` references `users.user_id`, allowing one user to have multiple membership purchases.

`workout_classes.trainer_id` references `users.user_id`, allowing one trainer to manage multiple workout classes.

Entity Relationship Diagram



7. Database Integration

The application uses JDBC to communicate with PostgreSQL.

`DatabaseConnection` provides database connections, which are then used by the DAO classes. The DAOs use `PreparedStatement` for parameterized SQL queries.

Database operations are kept in DAO classes rather than being written directly inside menus or service classes.

8. Dependencies

The project uses Maven for dependency management.

The main external dependencies are:

PostgreSQL JDBC Driver — allows Java to connect to and communicate with PostgreSQL.

jBCrypt — provides BCrypt password hashing and password verification.

The exact dependency versions are defined in `pom.xml`.

9. Logging

The application uses Java's `java.util.logging` framework with a custom `AppLogger` class.

Logs are written to:

`app.log`

The application records the required events, including:

- System startup
- Failed login attempts
- Database transaction errors
- Admin actions

The generated `app.log` file is excluded through `.gitignore`.

Logging provides a persistent record of important application activity that can still be reviewed after the console application closes.

10. Admin File Export

The Admin menu includes an **Export Reports to File** option. The Admin can choose between two reports:

- **Merchandise Inventory Report** — displays current merchandise records, stock levels, individual item values, and total inventory value.
- **Membership Revenue Report** — displays current membership records and total membership revenue.

The export feature uses two classes:

- `MerchandiseReportExporter`
- `MembershipReportExporter`

Each exporter retrieves current data through the existing service and DAO layers rather than creating duplicate database queries.

The export flow is:

AdminMenu → Report Exporter → Service → DAO → PostgreSQL → .txt File

Reports are written to:

`/reports/`

The generated files are:

```
reports/  
├── merchandise_report.txt  
└── membership_revenue_report.txt
```

Before writing a report, the exporter checks whether the `/reports/` directory exists. If it does not exist, Java's `File` class and `mkdir()` are used to create it automatically.

The reports are written using `FileWriter`. A for-loop goes through the returned `ArrayList` and writes each record in a human-readable format.

`FileWriter` is opened using **try-with-resources**, which automatically closes the writer when the operation finishes. This prevents file resources from being left open and helps ensure report data is properly written.

11. Project Structure

```
src/main/java/com/gymmembership/  
├── Main.java  
├── database/  
├── logging/  
├── menus/  
├── user/  
├── membership/  
├── merchandise/  
├── workout/  
└── reports/  
  
docs/  
├── user_guide.md  
├── developer_guide.md  
└── individual-reports/  
  
reports/  
pom.xml
```

12. Setup Instructions

To run the project:

1. Clone the GitHub repository.
2. Open the project in IntelliJ IDEA.
3. Ensure JDK 17 or higher is configured.
4. Install and start PostgreSQL.
5. Create the project database.
6. Run the provided SQL setup script to create the required tables.
7. Add the required database credentials to `application.properties`.
8. Use Maven to install the required dependencies.
9. Run `Main.java`.

`application.properties` is excluded from Git to prevent database credentials from being committed to the repository.